



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251221
Nama Lengkap	RIZKY FEBRIANTO KRISTIANSYAH
Minggu ke / Materi	14 / Tipe Data Set

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: Materi

Pengenalan dan Mendefinisikan Set

Set adalah salah satu tipe data pada Python yang digunakan untuk menyimpan sekumpulan data yang semuanya unik, dan biasa dikenal juga dengan istilah himpunan. Isi dari Set disebut sebagai anggota (member). Anggota dari Set harus bersifat immutable, sehingga tipe data seperti integer, float, string, dan tuple dapat dimasukkan ke dalam Set, sedangkan list dan dictionary yang bersifat mutable tidak dapat menjadi anggota Set. Set sendiri bersifat mutable, artinya isi dari sebuah Set dapat ditambah maupun dikurangi. Karena itu Set tidak dapat dimasukkan ke dalam Set lain.

Untuk mendefinisikan Set, dapat dilakukan dengan dua cara yaitu menggunakan notasi kurung kurawal {} atau dengan fungsi bawaan set(). Contoh pendefinisian Set menggunakan notasi {} adalah sebagai berikut:

```
# dengan menggunakan {}
bilangan_genap = {2, 4, 6, 8, 10, 12}
bilangan_ganjil = {1, 3, 5, 7, 9, 11}

# dengan menggunakan fungsi set()
pernah_ke_bulan = set('Neil Armstrong', 'Buzz Aldrin')
```

Perlu diperhatikan bahwa untuk mendefinisikan Set kosong, tidak dapat menggunakan notasi {} karena notasi tersebut akan menghasilkan dictionary kosong. Untuk Set kosong harus menggunakan fungsi set() seperti contoh berikut:

```
# dengan fungsi set()
pernah_ke_mars = set() # menghasilkan set kosong

data = {} # ini akan menghasilkan dictionary kosong
```

Pengaksesan Set

Set tidak memiliki indeks, sehingga anggota-anggota dari sebuah Set tidak dapat diakses secara langsung menggunakan indeks seperti pada list atau tuple. Untuk mengakses seluruh isi Set, dapat dilakukan menggunakan perulangan for. Selain itu, fungsi len() dapat digunakan untuk mengetahui jumlah anggota yang ada di dalam sebuah Set. Perlu diperhatikan bahwa urutan output yang dihasilkan bisa berbeda dengan urutan saat Set dideklarasikan, karena pada Set posisi anggota tidaklah penting dan tidak ada indeks yang mengaturnya.

Set bersifat mutable, sehingga isinya bisa bertambah maupun berkurang. Untuk menambahkan anggota baru ke dalam Set dapat menggunakan fungsi `add()`. Fungsi `add()` sudah memiliki mekanisme pengecekan duplikasi secara otomatis, sehingga apabila anggota yang akan ditambahkan sudah ada di dalam Set, maka pemanggilan fungsi `add()` tidak akan mengubah isi Set. Berikut ini adalah contoh program yang mendemonstrasikan pengaksesan dan penambahan anggota Set:

```
plat_nomor = set()
plat_nomor.add('AB 1890 XA')
plat_nomor.add('AD 6810 MT')
print(len(plat_nomor)) # output: 2
plat_nomor.add('AB 1890 XA') # tidak bertambah, sudah ada
for plat in plat_nomor:
    print(plat)
```

Untuk menghapus anggota dari sebuah Set, tersedia empat fungsi yaitu `discard()`, `remove()`, `pop()`, dan `clear()`. Fungsi `discard()` menghapus anggota yang disebutkan tanpa menghasilkan error meskipun anggota tersebut tidak ada. Fungsi `remove()` juga menghapus anggota yang disebutkan, namun akan menghasilkan error apabila anggota tidak ditemukan. Fungsi `pop()` mengambil dan mengeluarkan salah satu anggota secara acak dari Set, dan akan menghasilkan error apabila Set dalam kondisi kosong. Fungsi `clear()` akan menghapus seluruh anggota yang ada di dalam Set sekaligus.

discard()	remove()	pop()	clear()
Menghapus satu elemen yang disebutkan	Menghapus satu elemen yang disebutkan	Mengambil salah satu dan menghapusnya dari set (tidak tentu)	Menghapus seluruh elemen di dalam set
Tidak ada error	Muncul error jika elemen yang dihapus tidak ada	Error jika set kosong	Tidak ada error

Contoh penerapannya adalah sebagai berikut:

```

bilangan_prima = {13, 23, 7, 29, 11, 5}
# hapus 5 dari set tersebut
bilangan_prima.remove(5)
print(bilangan_prima)
# hapus 97 (tidak ada)
bilangan_prima.discard(97)
print(bilangan_prima)
# ambil dan hapus salah satu
bilangan = bilangan_prima.pop()
print(bilangan)
print(bilangan_prima)
# kosongkan set
bilangan_prima.clear()
print(bilangan_prima)

```

Output yang dihasilkan dari program itu adalah:

```

{5, 7, 11, 13, 23, 29}      # awal
{7, 11, 13, 23, 29}        # setelah 5 dihapus. Perhatikan urutan berubah
{7, 11, 13, 23, 29}        # 97 tidak ada, sehingga Set tidak berubah
7                            # fungsi pop() mengeluarkan 7
{11, 13, 23, 29}           # setelah 7 keluar dari Set
set()                       # setelah isi dari Set dihapus semua

```

Untuk mengubah nilai anggota di dalam Set tidak dapat dilakukan secara langsung. Cara yang harus dilakukan adalah dengan terlebih dahulu menghapus anggota yang ingin diubah menggunakan `remove()`, kemudian menambahkan anggota baru dengan nilai yang diinginkan menggunakan `add()`. Berikut contoh operasi penggantian anggota Set:

```

ikan = set(['koi', 'koki', 'kembung', 'salmon'])
ikan.remove('koi')
ikan.add('teri')
print(ikan) # {'teri', 'salmon', 'kembung', 'koki'}

```

Operasi-operasi pada Set

Operasi-operasi pada Set merupakan operasi himpunan yang melibatkan dua Set atau lebih. Python menyediakan empat operator utama pada Set yaitu Union, Intersection, Difference, dan Symmetric Difference. Masing-masing operator tersebut dapat digunakan menggunakan simbol operator maupun melalui pemanggilan fungsi yang tersedia.

Operator Union

Operator Union digunakan untuk menggabungkan dua Set menjadi satu Set baru yang berisi semua anggota dari kedua Set tersebut. Karena sifat unik dari Set, anggota yang sama hanya akan

muncul satu kali di dalam hasil Union. Operator Union dapat menggunakan simbol `|` atau fungsi `union()`. Berikut contoh penggunaannya:

```
merek_hp = {'Samsung', 'Apple', 'Xiaomi', 'Sony'}
merek_ac = {'LG', 'Samsung', 'Panasonic', 'Daikin', 'Sony'}
gabungan = merek_hp | merek_ac
# bisa juga: gabungan = merek_hp.union(merek_ac)
print(gabungan)
# Output: {'Xiaomi', 'Sony', 'LG', 'Daikin', 'Samsung', 'Apple', 'Panasonic'}
```

Pada contoh tersebut, 'Samsung' dan 'Sony' muncul di kedua Set namun hanya muncul satu kali pada hasil Union karena anggota Set harus unik.

Operator Intersection

Operator Intersection menghasilkan irisan dari dua Set, yaitu Set baru yang hanya berisi anggota-anggota yang ada di dalam kedua Set sekaligus. Operator Intersection dapat menggunakan simbol `&` atau fungsi `intersection()`. Berikut contoh penggunaannya:

```
renang = {'siti', 'mail', 'ikhshan', 'upin', 'ipin'}
tenis = {'joko', 'mail', 'ipin', 'upin', 'tejo'}
renang_tenis = renang & tenis
print(renang_tenis)
# Output: {'upin', 'ipin', 'mail'}
```

Hasil dari operasi intersection tersebut adalah mail, upin, dan ipin, yaitu anggota-anggota yang terdaftar di kedua Set renang maupun Set tenis sekaligus.

Operator Difference

Operator Difference menghasilkan Set baru yang berisi anggota-anggota yang ada di Set pertama tetapi tidak ada di Set kedua. Dengan kata lain, hasil dari $A - B$ adalah anggota A yang tidak termasuk dalam B. Operator Difference dapat menggunakan simbol `-` atau fungsi `difference()`. Berikut contoh penggunaannya:

```
english = {'desi', 'tono', 'evan', 'miko', 'takashi', 'chaewon'}
korean = {'chaewon', 'yeona', 'erika', 'miko'}
only_korean = korean - english
print(only_korean) # Output: {'erika', 'yeona'}
only_english = english - korean
print(only_english) # Output: {'takashi', 'desi', 'evan', 'tono'}
```

Pada contoh di atas, `only_korean` berisi anggota yang hanya bisa berbahasa Korea (tidak terdaftar di Set `english`), sedangkan `only_english` berisi anggota yang hanya bisa berbahasa English (tidak terdaftar di Set `korean`).

Operator Symmetric Difference

Operator Symmetric Difference menghasilkan Set baru yang merupakan gabungan dari dua Set tetapi tidak termasuk irisannya. Dengan kata lain, hasilnya adalah semua anggota yang hanya ada di salah satu Set saja. Operator Symmetric Difference dapat menggunakan simbol ^ atau fungsi `symmetric_difference()`. Secara matematis, hasil operasi ini sama dengan union dikurangi intersection: $(A \cup B) - (A \cap B)$. Berikut contoh penggunaannya:

```
english = {'desi', 'tono', 'evan', 'miko', 'takashi', 'chaewon'}
korean = {'chaewon', 'yeona', 'erika', 'miko'}
one_language = english ^ korean
print(one_language)
# Output: {'yeona', 'tono', 'desi', 'erika', 'evan', 'takashi'}
```

Hasil tersebut berisi semua orang yang hanya bisa berbicara dalam satu bahasa saja, di mana 'miko' dan 'chaewon' yang merupakan irisan (bisa kedua bahasa) tidak termasuk dalam hasil Symmetric Difference.

BAGIAN 2: LATIHAN MANDIRI

Link Github repository: https://github.com/ifgodwillsit/71251221_febrian

Latihan 14.1

```
n = int(input('Masukkan jumlah kategori: '))
data_aplikasi = {}
for i in range(n):
    nama_kategori = input('Masukkan nama kategori: ')
    jumlah = int(input(f"Berapa jumlah aplikasi di kategori {nama_kategori}: "))
    aplikasi = []
    for j in range(jumlah):
        nama_aplikasi = input('Nama aplikasi: ')
        aplikasi.append(nama_aplikasi)
    data_aplikasi[nama_kategori] = set(aplikasi)

daftar_set = list(data_aplikasi.values())

hanya_satu = set()
for i in range(len(daftar_set)):
    gabungan_lain = set()
    for j in range(len(daftar_set)):
        if i != j:
            gabungan_lain = gabungan_lain | daftar_set[j]
    hanya_satu = hanya_satu | (daftar_set[i] - gabungan_lain)
print('Aplikasi yang hanya muncul di satu kategori:', hanya_satu)

if n > 2:
    tepat_dua = set()
    for i in range(len(daftar_set)):
        for j in range(i + 1, len(daftar_set)):
            irisan = daftar_set[i] & daftar_set[j]
            for k in range(len(daftar_set)):
                if k != i and k != j:
                    irisan = irisan - daftar_set[k]
            tepat_dua = tepat_dua | irisan
    print('Aplikasi yang muncul tepat di dua kategori:', tepat_dua)
```

Gambar 2.1: Sourcecode Latihan 14.1

Konsep dari latihan ini adalah mengembangkan program kategori aplikasi Play Store dari Kegiatan Praktikum dengan menambahkan dua fitur baru. Untuk menampilkan aplikasi yang hanya muncul di satu kategori, digunakan operasi difference (-) antara Set setiap kategori dengan gabungan (|) semua Set dari kategori lainnya. Hasilnya adalah anggota yang eksklusif hanya ada di satu kategori. Untuk menampilkan aplikasi yang muncul tepat di dua kategori, digunakan kombinasi intersection (&) antar pasangan Set kemudian dikurangi (-) dengan Set-Set lain yang tersisa. Proses ini memastikan hanya aplikasi yang berada tepat di dua kategori saja yang tertampil.

```

Masukkan jumlah kategori: 3
Masukkan nama kategori: Game
Berapa jumlah aplikasi di kategori Game: 3
Nama aplikasi: FF
Nama aplikasi: MLBB
Nama aplikasi: WarThunder
Masukkan nama kategori: Kesehatan
Berapa jumlah aplikasi di kategori Kesehatan: 3
Nama aplikasi: MBG
Nama aplikasi: Alodoc
Nama aplikasi: MLBB
Masukkan nama kategori: Edukasi
Berapa jumlah aplikasi di kategori Edukasi: 4
Nama aplikasi: Ruangguru
Nama aplikasi: FF
Nama aplikasi: MLBB
Nama aplikasi: MyBukuGua
Aplikasi yang hanya muncul di satu kategori: {'Alodoc', 'MyBukuGua', 'MBG', 'WarThunder', 'Ruangguru'}
Aplikasi yang muncul tepat di dua kategori: {'FF'}

```

Gambar 2.2: Output Latihan 14.1

Latihan 14.2

```

print("=== List ke Set ===")
data_list = eval(input("Masukkan list (contoh: [1, 2, 3, 2, 4]): "))
print("List awal      :", data_list)
data_set = set(data_list)
print("Set hasil konversi:", data_set)
print()

print("=== Set ke List ===")
data_set2 = eval(input("Masukkan set (contoh: {'apel', 'mangga', 'pisang'}): "))
print("Set awal          :", data_set2)
data_list2 = list(data_set2)
print("List hasil konversi:", data_list2)
print()

print("=== Tuple ke Set ===")
data_tuple = eval(input("Masukkan tuple (contoh: (10, 20, 30, 20)): "))
print("Tuple awal         :", data_tuple)
data_set3 = set(data_tuple)
print("Set hasil konversi:", data_set3)
print()

print("=== Set ke Tuple ===")
data_set4 = eval(input("Masukkan set (contoh: {'Python', 'Java', 'C++'}) : "))
print("Set awal           :", data_set4)
data_tuple2 = tuple(data_set4)
print("Tuple hasil konversi:", data_tuple2)

```

Gambar 2.3: Sourcecode Latihan 14.2

Konsep dari latihan ini adalah memperlihatkan cara konversi antar tipe data koleksi di Python. Konversi dilakukan menggunakan fungsi bawaan `set()`, `list()`, dan `tuple()`. Hal yang menarik terjadi saat melakukan konversi dari List atau Tuple ke Set, di mana elemen-elemen yang duplikat secara otomatis akan dihilangkan karena sifat unik dari Set. Sebaliknya, saat mengkonversi Set ke List atau Tuple, urutan elemen tidak dapat dijamin sama karena Set tidak memiliki indeks. Simpel aja sih, tinggal mengubah tipe data dari data hasil input ke tipe data yang diinginkan. Program ini juga memperlihatkan perubahan tampilan data sebelum dan sesudah konversi.

```
=== List ke Set ===
Masukkan list (contoh: [1, 2, 3, 2, 4]): [1, 2, 3, 2, 4]
List awal      : [1, 2, 3, 2, 4]
Set hasil konversi: {1, 2, 3, 4}

=== Set ke List ===
Masukkan set (contoh: {'apel', 'mangga', 'pisang'}): {'apel', 'mangga', 'pisang'}
Set awal      : {'pisang', 'apel', 'mangga'}
List hasil konversi: ['pisang', 'apel', 'mangga']

=== Tuple ke Set ===
Masukkan tuple (contoh: (10, 20, 30, 20)): (10, 20, 30, 20)
Tuple awal    : (10, 20, 30, 20)
Set hasil konversi: {10, 20, 30}

=== Set ke Tuple ===
Masukkan set (contoh: {'Python', 'Java', 'C++'}): {'Python', 'Java', 'C++'}
Set awal      : {'Python', 'Java', 'C++'}
Tuple hasil konversi: ('Python', 'Java', 'C++')
```

Gambar 2.4: Output Latihan 14.2

Latihan 14.3

```
def baca_kata(nama_file):
    kata_set = set()
    try:
        with open(nama_file, 'r') as f:
            for baris in f:
                kata_kata = baris.lower().split()
                for kata in kata_kata:
                    kata_bersih = kata.strip('.,!?:;"-()[]')
                    if kata_bersih:
                        kata_set.add(kata_bersih)
            return kata_set
    except FileNotFoundError:
        print(f'Error: File "{nama_file}" tidak ditemukan.')
        return None

file1 = input('Masukkan nama file pertama: ')
file2 = input('Masukkan nama file kedua: ')

set1 = baca_kata(file1)
set2 = baca_kata(file2)

if set1 is not None and set2 is not None:
    kata_bersama = set1 & set2
    print(f'\nKata-kata yang muncul pada kedua file:')
    for kata in sorted(kata_bersama):
        print(kata)
    print(f'\nTotal: {len(kata_bersama)} kata')
```

Gambar 2.5: Sourcecode Latihan 14.3

Konsep dari latihan ini adalah membaca dua file teks dan mencari kata-kata yang muncul di keduanya menggunakan operasi intersection pada Set. Program mendefinisikan fungsi `baca_kata()` yang menerima nama file sebagai parameter dan mengembalikan Set berisi semua kata unik dari file tersebut. Di dalam fungsi tersebut, setiap baris dibaca, dikonversi ke huruf kecil menggunakan `lower()`, kemudian dipecah menjadi kata-kata menggunakan `split()`. Setiap kata juga dibersihkan dari tanda baca menggunakan `strip()` sebelum dimasukkan ke dalam Set. Penanganan error menggunakan `try-except` diimplementasikan untuk menangkap `FileNotFoundError` apabila file yang dimasukkan user tidak ditemukan. Setelah kedua Set terbentuk, operasi intersection (`&`) digunakan untuk mendapatkan kata-kata yang muncul di kedua file sekaligus, dan hasilnya ditampilkan secara terurut menggunakan `sorted()`.

```
temporary.txt
1  when yah
2  my bini is where
```

Gambar 2.6: File temporary.txt

```
temp.txt
1  my bini gua kemana yah
2  when yah
3  sangat amat super duper mencari
```

Gambar 2.7: File temp.txt

```
Masukkan nama file pertama: temp.txt
Masukkan nama file kedua: temporary.txt

Kata-kata yang muncul pada kedua file:
bini
my
when
yah
```

Gambar 2.8: Output Latihan 14.3